QualiAdept

Sesiunea 7 - Tipizarea statică

Capitolul 1: Introducere în TypeScript - De ce a evoluat JavaScript-ul? 3
Obiective de învățare 3
1.1 Ce este TypeScript (TS)? 3
1.2 Ce înseamnă, de fapt, "Tipizarea" (Typing)? 4
1.3 Scurtă Istorie: De ce Microsoft și de ce acum? 5
1.4 Tipizare Dinamică (JS) vs. Tipizare Statică (TS): Exemple Practice 5
1.5 De ce folosește Playwright TypeScript? 6
1.6 💡 Știați că...? 7
1.7 🧠 Întrebare Capcană de Interviu (Interview Challenge) 7
1.8 🤖 Mentalitatea de Inginer în Era AI: Contextul "Hallucination-Free" 8
Capitolul 2: Primele linii de cod - Tipuri primitive, any, unknown și void 9
Obiective de învățare 9
2.1 Unde rulăm codul? Magia Playwright vs. Realitatea Browserelor 9
2.2 Anotarea Tipurilor Primitive (Type Annotations) 10
Magia TypeScript: Infernța de tip (Type Inference) 11
2.3 "Băiatul Rău" al TypeScript-ului: Tipul any 11
2.4 Alternativa Inteligentă: Tipul unknown 12
2.5 Funcțiile și Tipul void (Nimic) 13
2.6 🧠 Challenge de Interviu (Întrebare Tehnică) 13
2.7 🤖 Mentalitatea de Inginer în Era AI: "Lenea" Asistenților Generativi 14
Capitolul 3: Structuri Avansate - Interfețe (Interfaces) și Tipuri Custom 15
Obiective de învățare 15
3.1 Problema: Obiectele haotice din JavaScript 15
3.2 Tipurile Custom (Type Aliases) și Tipurile Union 16
3.3 Standardul de Aur pentru Date: Interfețele (Interfaces) 16
3.4 Magia Interfețelor: ? (Opțional) și readonly 17
3.5 Cum folosim Interfețele în Automatizarea cu Playwright? 18
3.6 🧠 Challenge de Interviu (Întrebare Tehnică) 18

3.7 🤖 Mentalitatea de Inginer în Era AI: "Prompting pe bază de Contract" 19
Capitolul 4: Centrul de Comandă - Configurarea tsconfig.json 19
Obiective de învățare 20
4.1 De ce a fost nevoie de tsconfig.json? (Contextul Istoric) 20
4.2 Laborator Interactiv: Cum schimbă flag-urile codul tău 20
Misiunea 1: target (Mașina Timpului) 21
Misiunea 2: noImplicitAny (Detectorul de lene) 21
Misiunea 3: strictNullChecks (Salvatorul absolut în QA Automation) 22
Regula Supremă: "strict": true 23
4.3 Exemplu de fișier tsconfig.json minimalist pentru Playwright 23
4.4 💡 Știați că...? 23
4.5 🧠 Challenge de Interviu (Întrebare Tehnică) 24
4.6 🤖 Mentalitatea de Inginer în Era AI 24
Capitolul 5: Concepte Avansate (Bonus) - Afirmarea Tipurilor (Type Assertions) 25
Obiective de învățare 25
5.1 Problema: Când TypeScript știe mai puține decât tine 26
5.2 Salvarea: Cuvântul cheie as (Type Casting) 26
5.3 Pericolul ascuns: Minținea Compilatorului 27
5.4 Legătura cu Playwright: Executarea codului în Browser 28
5.5 🤖 Mentalitatea de Inginer în Era AI: Asertările Oarbe 28
Capitolul 6: Tema Sesiunii 7 & Extra Practice (Spre nivelul de Senior) 29
6.1 Tema pentru Acasă (Proiect Sesiunea 7) & Platforma QualiAdept Cloud Evaluation 29
🏢 Scenariul de Business (Task-ul de business): 30
📋 Cerințe Tehnice (Acceptance Criteria pentru platforma de validare): 30
💡 Analogie QA Automation (De ce este vital pentru Playwright): 31
6.2 Provocarea Bonus (Opțional): Siguranță absolută cu unknown 31
6.3 🔑 Cheile de Rezolvare (Soluții & Explicații) 32
Workbook Sesiunea 7 - TypeScript Core 34
📍 Exercițiul 1: Funcții, Primitive și Type Inference 34
📍 Exercițiul 2: Union Types & Literal Types (Opțiuni Restricționate) 35
📍 Exercițiul 3: Siguranța cu unknown (Type Guards) 36
📍 Exercițiul 4: Interfețe și Array-uri (Manipularea datelor) 37
📍 Exercițiul 5: Clase și Încapsulare (Page Object Model) 37

📍 Exercițiul 6: Moștenirea (Extends & Super) 39
📍 Exercițiul 7: Manipularea DOM-ului și Type Casting (as) 40

Capitolul 1: Introducere în TypeScript - De ce a evoluat JavaScript-ul?

Obiective de învățare

- 🎯 Înțelegi conceptul de "Tipizare" și diferența fundamentală dintre un limbaj cu tipizare dinamică (JavaScript) și unul cu tipizare statică (TypeScript).
- 🎯 Afli povestea apariției TypeScript și motivele pentru care gigantul Microsoft l-a creat.
- 🎯 Răspunzi corect la scenarii de interviu tehnic privind execuția TypeScript în browser.
- 🎯 Descoperi de ce Playwright și întreaga industrie de Enterprise Software au adoptat TypeScript ca standard absolut.

1.1 Ce este TypeScript (TS)?

În modulele anterioare, ai învățat **JavaScript (JS)**. Acesta este limbajul nativ al browserelor, incredibil de flexibil, dar uneori... *prea flexibil*. Această flexibilitate duce frecvent la bug-uri care apar abia când utilizatorul apasă un buton (la *runtime*).

TypeScript NU este un limbaj complet nou. Este un **"Superset"** (o supramulțime) a JavaScript-ului, creat și menținut de Microsoft.

Gândește-te la TypeScript ca la *JavaScript cu reguli stricte, un corector automat și un sistem avansat de radar pentru erori*.

- **Regula de aur:** Orice cod valid de JavaScript este automat și un cod valid de TypeScript. Dar TypeScript adaugă straturi de siguranță peste el.
- **Cum rulează?** Browserele (Chrome, Firefox) și Node.js **NU știu** să citească TypeScript.

Așadar, codul TS trebuie tradus (compilat/transpilat) înapoi în JavaScript curat înainte de a fi rulat. Playwright face această traducere pentru tine automat în fundal!

1.2 Ce înseamnă, de fapt, "Tipizarea" (Typing)?

Pentru a înțelege de ce s-a inventat TypeScript, trebuie să definim conceptul de "Tipizare".

Tipizarea reprezintă setul de reguli prin care un limbaj de programare asignează o categorie (un "tip de date" precum String, Number, Boolean) variabilelor tale și cât de strict impune aceste reguli.

- **Tipizarea Dinamică (abordarea JavaScript):** Calculatorul "ghicește" tipul de date "din mers", abia în momentul în care codul rulează (la *runtime*). O cutie (let) poate să conțină un număr pe o linie de cod, iar 5 rânduri mai jos să primească un text, fără ca limbajul să se plângă. Este un sistem rapid pentru prototipuri, dar periculos la scalare.
- **Tipizarea Statică (abordarea TypeScript / Java / C#):** Tu, ca programator, declari tipul de date *în avans*, în momentul scrierii codului. Dacă declari că o cutie ține numere, ea devine strictă. Dacă încerci să pui un text în ea, editorul îți va bloca scriptul (la *compile-time*) și îți va da eroare înainte să apuci să rulezi testul.

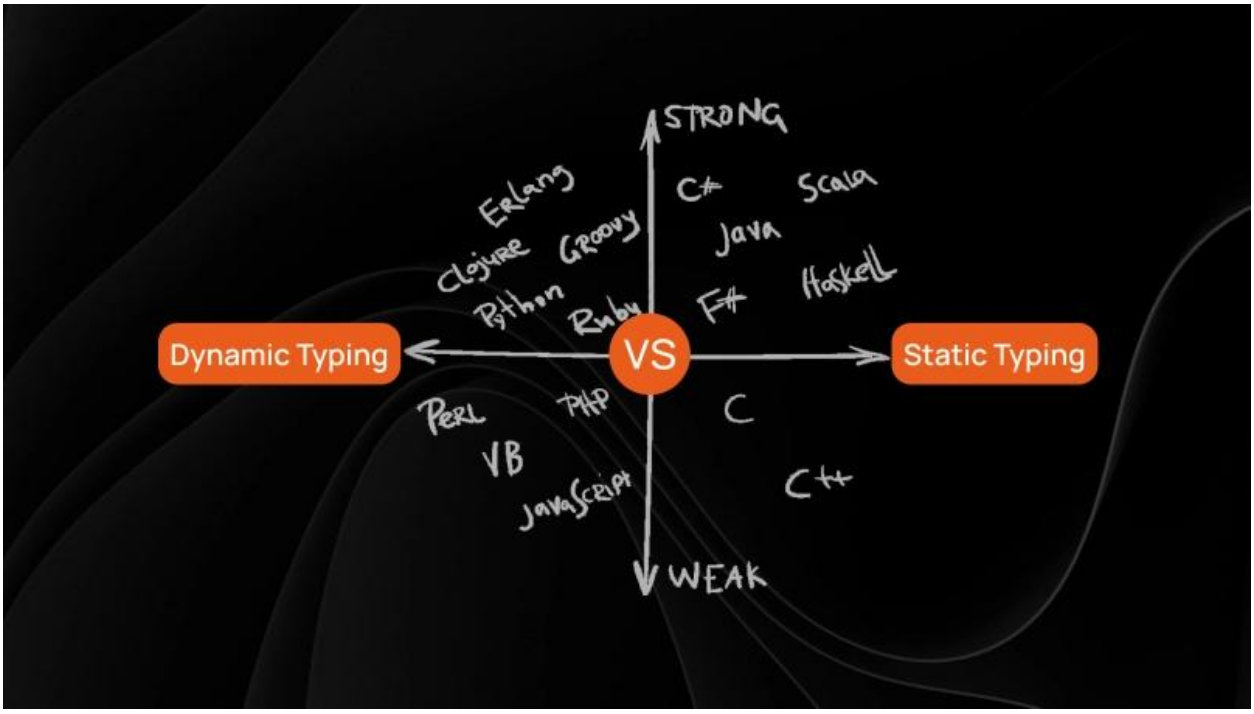


Fig. 1 - Dynamic Typing vs Static Typing

1.3 Scurtă Istorie: De ce Microsoft și de ce acum?

> 📝 Limbajul JavaScript a fost creat în 1995 în doar 10 zile, cu scopul de a face mici animații pe site-uri web (cum ar fi zăpada care cade pe ecran). Nimeni nu s-a gândit că va fi folosit pentru sisteme uriașe.
> Apoi a venit era aplicațiilor web masive (Web 2.0). Microsoft încerca să construiască suita **Office 365** direct în browser. Aveau milioane de linii de cod JavaScript. Din cauza flexibilității JS (tipizarea dinamică), o echipă schimba o variabilă din Număr în Text, iar altă echipă din capătul celălalt al lumii avea o funcție care se baza matematic pe acea variabilă. Rezultatul? Sistemul crăpa complet.
>
> Pentru a salva situația, în **2012**, Microsoft (conduși de **Anders Hejlsberg**, geniul din spatele limbajului C#) a lansat TypeScript. Scopul nu a fost să distrugă JavaScript, ci să îi aducă rigoarea și disciplina (Tipizarea Statică) din limbajele Enterprise (precum Java sau C#), permițând companiilor să construiască aplicații uriașe fără frica de a introduce "bug-uri silențioase".

1.4 Tipizare Dinamică (JS) vs. Tipizare Statică (TS): Exemple Practice

Cea mai mare diferență între cele două este **momentul** în care afli că ai făcut o greșeală.

Exemplul 1: Schimbarea tipului de date (Variabile)

- **JS (Dinamic):** Calculatorul nu se plânge până când nu rulează efectiv codul.

```
// JS: Nu există protecție
let pret = 100;
pret = "O sută";      // Am pus text în loc de număr. JS zice "E OK!"
let reducere = pret - 20;
console.log(reducere); // La rulare primim NaN (Not a Number)! Testul pică!
```

- **TS (Static):** Erorile îți sunt semnalate direct în editorul tău (VS Code), *înainte* să rulezi testul!

```
// TS: Protecție activă
let pret: number = 100; // Declarăm clar: Această cutie acceptă DOAR numere!
```

```
pret = "O sută";
// ❌ EROARE INSTANTANEE ÎN VS CODE (Subliniat cu roșu):
// "Type 'string' is not assignable to type 'number'."
```

Exemplul 2: Parametrii Funcțiilor

Să presupunem că ai o funcție care dă click pe al "X"-lea element dintr-o listă.

- **JS:** Funcția este oarbă.

```
function clickPeElementul(index) {
    // Dacă primești text în loc de număr, logica internă se strică.
}
clickPeElementul("doi"); // JS va accepta asta și va da eroare mai târziu.
```

- **TS:** Definim un contract strict (Signature).

```
function clickPeElementul(index: number) { ... }

clickPeElementul("doi"); // ❌ Eroare: Argument of type 'string' is not
assignable to parameter of type 'number'.
```

Exemplul 3: Lucrul cu Obiecte (Auto-completare / IntelliSense)

- **JS:** Dacă ai un obiect utilizator, VS Code nu știe ce conține. Dacă încerci să citești utilizator.nume dar de fapt proprietatea se numea utilizator.numeComplet, JS va returna undefined în tăcere. Testul tău va avea un fals pozitiv sau va eșua misterios.
- **TS:** Datorită tipizării statice (vom învăța despre *Interfețe* în Capitolul 2), editorul știe perfect ce conține obiectul. Când tastezi utilizator., VS Code îți va deschide un meniu autocompletat doar cu proprietățile valide, economisindu-ți ore întregi de căutări și debugging.

1.5 De ce folosește Playwright TypeScript?

Dacă TS necesită să scriem mai mult cod (să declarăm tipurile), de ce companiile mari și Playwright îl preferă?

- **Prinderea Bug-urilor la scrierea codului (Compile Time):** Ca inginer de automatizare, nu vrei să lansezi 500 de teste în CI/CD, să aștepți 20 de minute, doar ca să afli că ai scris usrname în loc de username.
- **Mentenanța ușoară:** Când interfața se schimbă și dezvoltatorii Frontend modifică structura datelor, TS îți va sublinia instant cu roșu absolut TOATE cele 50 de teste din proiectul tău

care trebuie actualizate, fără să fii nevoit să le rulezi orbește.

- **Cod Autodocumentat:** Când citești o funcție TypeScript scrisă de un coleg, știi exact ce fel de date așteaptă și ce returnează, fără să fii nevoit să îi citești logica internă.

1.6 💡 Știați că...?

- **TypeScript nu există în producție:** Deși dezvoltatorii scriu și testează codul în TS, înainte ca aplicația să fie lansată efectiv pe internet, tot codul TS trece printr-un compilator (ex: tsc sau Babel) care îi șterge absolut toate regulile și tipurile, lăsând în urmă un fișier de JavaScript curat și banal. Toată "magia" TypeScript trăiește doar pe laptopul tău (în timpul fazei de Dezvoltare/Testare)!
- **Adopție Masivă:** Conform sondajelor anuale StackOverflow, TypeScript este constant clasat în top 3 "Cele mai iubite limbaje de programare", iar astăzi aproape toate librăriile majore (inclusiv React) se bazează pe el.
- **A prins NASA:** Framework-urile folosite de NASA pentru panourile de control ale roverelor de pe Marte folosesc variante stricte de JavaScript tipizat, tocmai pentru a preveni crash-uri neașteptate.

1.7 🧠 Întrebare Capcană de Interviu (Interview Challenge)

🧠 Aceasta este o întrebare preferată la interviurile pentru roluri de QA Automation Engineer:

Intervievatorul: *"Observ că ai scris framework-ul de Playwright în TypeScript. Spune-mi, te rog: dacă testele ar fi fost scrise în JavaScript simplu, s-ar fi executat mai lent în browserul Chrome decât cele scrise în TypeScript? TypeScript face testele mai rapide la execuție?"*

Răspunsul tău de inginer:

"Nu, execuția în browser durează absolut la fel. Browserele web (ca și Chrome) nu pot înțelege

*și nu pot rula TypeScript. Playwright compilează codul meu TypeScript în JavaScript curat în fundal, chiar înainte de a lansa browserul. Avantajul TypeScript nu este viteza de rulare (runtime), ci viteza și siguranța **scrierii** codului (compile-time). TS mă ajută pe mine, ca inginer, să descopăr erorile direct în editor, înainte ca testul să apuce să fie executat de browser."*

1.8 🤖 Mentalitatea de Inginer în Era AI: Contextul "Hallucination-Free"

🌓 QA vs. AI: De ce iubește AI-ul TypeScript?

- **Context:** Modelele de AI generativ (ca ChatGPT, GitHub Copilot) funcționează predicând următorul cuvânt logic pe baza contextului pe care i-l oferi.
- **Problema cu JS:** Dacă îi ceri AI-ului *"scrie un test care validează un produs"*, în JavaScript el nu știe ce proprietăți are "produsul". Ar putea ghici că produsul are produs.titlu, dar aplicația ta poate folosește produs.nume. AI-ul începe să "halucineze" cod care pare corect, dar care va pica testul.
- **Avantajul TypeScript:** În TS, tu definești interfețele strict (vom învăța în Capitolul 3). Când îi ceri AI-ului să scrie testul, el citește tipizarea TS și *"vede"* clar că aplicația ta folosește produs.nume. Astfel, acuratețea codului generat de asistenții AI pe proiecte TypeScript crește de la 60% la peste 95%!
- **Viitorul Tău:** Adoptând TypeScript, nu doar că scrii cod mai sigur, dar îi oferi robotului tău AI ghidajele (guardrails) necesare pentru a te ajuta la capacitate maximă, transformându-te dintr-un simplu tester într-un "AI-Assisted Test Architect".

Capitolul 2: Primele linii de cod - Tipuri primitive, any, unknown și void

Obiective de învățare

- 🎯 Descoperi mediul de testare (TypeScript Playground) pentru a scrie TS direct în browser.
- 🎯 Înțelegi cum se adaugă tipuri de date variabilelor folosind "Type Annotations" (Anotări de tip).
- 🎯 Înveți să te bazezi pe "Type Inference" (Infernța de tip), lăsând TypeScript să deducă singur tipurile.
- 🎯 Descoperi diferența esențială dintre any (inamicul calității) și unknown (abordarea defensivă) prin exemple rulate live.
- 🎯 Utilizezi tipul void pentru a scrie funcții de testare curate.

2.1 Unde rulăm codul? Magia Playwright vs.

Realitatea Browserelor

Spre deosebire de modulele trecute, avem o problemă tehnică majoră: **Browserele web (Chrome, Firefox, Safari) NU știu să citească TypeScript.** Ele cunosc doar limbajele de bază: HTML, CSS și JavaScript curat.

Dacă încerci să deschizi consola Chrome (F12) și să scrii let varsta: number = 30, vei primi o eroare de sintaxă instantanee, deoarece browserul nu recunoaște semnul : number. Codul TypeScript trebuie întotdeauna **compilat** (tradus și curățat de tipuri) într-un fișier JavaScript înainte de a fi executat.

🚀 **De ce iubește industria Playwright-ul? (The Out-of-the-box Magic)**

În mod tradițional, pentru a face această traducere, programatorii trebuiau să instaleze compilatoare complicate și să ruleze manual comenzi în terminal (precum tsc) de fiecare dată când modificau o linie de cod.

Playwright a revoluționat testarea pentru că **face absolut totul pentru noi în fundal!** Când vei scrie primul tău test Playwright într-un fișier .ts și vei apăsa butonul de "Run", framework-ul va compila codul tău TypeScript în JavaScript în doar câteva milisecunde, fără să îți ceară nicio configurare manuală. Tu scrii strict, Playwright rulează rapid!

Până când vom instala Playwright, pentru a face exercițiile din acest modul, vom folosi simulatorul oficial creat de Microsoft care face această compilare vizual pentru noi: **TypeScript Playground**.

🤓 ❓ **Setează-ți mediul de lucru:**

- Deschide un tab nou în browser și accesează: **www.typescriptlang.org/play**
- În panoul din stânga, vei scrie codul TypeScript.
- Observă magia: în panoul din dreapta, vezi în timp real cum codul tău este tradus în JavaScript banal! Toate erorile de tipizare îți vor fi subliniate cu roșu în stânga.

2.2 Anotarea Tipurilor Primitive (Type Annotations)

Sintaxa de bază a TypeScript-ului implică adăugarea caracterului "două puncte" (:) urmat de tipul de date, exact după numele variabilei sau parametrului.

```
// Sintaxa: let numeVariabila: Tip = valoare;

let username: string = "admin_qa";
let timeoutTest: number = 5000;
let esteLogat: boolean = true;
```

🤓 ❓ **Testează în Playground:**

Copiază liniile de mai sus în TS Playground. Apoi, pe un rând nou, încearcă să scrii: timeoutTest = "cinci secunde";.

Vei vedea instantaneu cum codul este subliniat cu roșu. Ai fost protejat de un bug înainte măcar să rulezi aplicația!

Magia TypeScript: Infernța de tip (Type Inference)

Trebuie să scriem mereu : string sau : number? **Nu!**

TypeScript este un limbaj extrem de inteligent. Dacă îi dai o valoare unei variabile chiar în momentul în care o declari, TS "ghicește" automat tipul și îl blochează pe acela.

```
// Așa scriu profesioniștii (mai curat, mai scurt)
let adresaURL = "https://platforma.ro";
// ^ TypeScript a văzut textul și a aplicat tăcut ": string" în fundal.

// adresaURL = 100; // Dacă decomentezi asta, dă eroare!
```

***Regula de aur**:* Folosește anotarea manuală (: tip) doar atunci când declari o variabilă goală, pe care o vei umple mai târziu, sau când definești parametrii unei funcții.

2.3 "Băiatul Rău" al TypeScript-ului: Tipul any

Când Microsoft a creat TypeScript peste JavaScript, a trebuit să lase o "portiță de scăpare" pentru a permite dezvoltatorilor să migreze codul vechi mai ușor. Acea portiță se numește **any** (orice).

Când folosești any, îi spui compilatorului TypeScript: *"Închide ochii, oprește radarul de erori, știu eu ce fac!"*. Practic, ai anulat absolut toate beneficiile limbajului și te-ai întors la JavaScript-ul sălbatic.

🤓 ❓ **Testează Pericolul în Playground:**

Lipește acest cod în TS Playground:

```
let variabilaMisterioasa: any = "Salut, sunt un text";

// Încercăm să folosim o metodă de matematică pe un text!
let calculGresit = variabilaMisterioasa.toFixed(2);
console.log(calculGresit);
```

Observi problema? TypeScript NU îți dă nicio eroare cu roșu. Editorul tău crede că totul e perfect. Dar dacă apesi butonul **"Run"** din Playground (care execută codul compilat în browser), consola îți va da un crash fatal: TypeError: variabilaMisterioasa.toFixed is not a function. any te-a mințit și a ascuns bug-ul!

În companiile de top, regula în Code Review este strictă: ***Fără any în proiect!***

2.4 Alternativa Inteligentă: Tipul unknown

Dacă nu e voie să folosim any, ce facem când facem o cerere API cu fetch și efectiv *nu știm* dinainte ce format de date ne va întoarce serverul? Aici intervine **unknown**.

Când o variabilă este unknown, TypeScript te lasă să pui orice în ea, dar **NU te lasă să o folosești** până când nu verifici (validezi) exact ce este!

🕵️‍♂️ ❓ **Testează Siguranța în Playground:**

Lipește acest cod:

```
let raspunsAPI: unknown = "200 OK";

// 1. Încearcă să folosești o metodă de String.
// Linia de mai jos VA DA EROARE în editor, pentru că TS te protejează!
// let textMare = raspunsAPI.toUpperCase();

// 2. Abordarea QA: Verificarea tipului (Type Narrowing)
if (typeof raspunsAPI === "string") {
    // În interiorul acestui 'if', TS știe 100% sigur că e un string.
    // Acum îți permite să folosești metodele specifice, ba chiar îți oferă
auto-completare!
    let textMare = raspunsAPI.toUpperCase();
    console.log(textMare);
}
```

Ca inginer de automatizare, **unknown** este excelent pentru testarea de API-uri, forțându-te să scrii asertări defensive și sigure înainte să procesezi datele.

2.5 Funcțiile și Tipul void (Nimic)

Am vorbit despre tipurile de date pentru variabile, dar și funcțiile returnează date. În JavaScript, dacă o funcție doar dă click pe un buton, dar nu are un cuvânt cheie return, ea returnează implicit undefined.

În TypeScript, o funcție care execută o acțiune dar nu dă nimic înapoi se marchează cu tipul **void** (vid / gol). Aceasta îi semnalează clar oricărui programator care îți citește codul că nu are sens să încerce să salveze rezultatul acelei funcții într-o variabilă.

```
// Această funcție returnează un Number (calculat)
const calculeazaSuma = (a: number, b: number): number => {
    return a + b;
};

// Această funcție doar execută un click, nu dă date înapoi. Tipul este :
void
const printeazaInConsola = (mesaj: string): void => {
    console.log("Mesajul este: " + mesaj);
    // Nu există 'return' aici!
};
```

2.6 🤓 Challenge de Interviu (Întrebare Tehnică)

> 🧠 **Intervievatorul:** "Mă uit peste Pull Request-ul tău. Ai folosit any pentru a prelua un răspuns de la API, pentru că serverul nostru este instabil. De ce ți-am cerut să schimbi în unknown? Care este diferența tehnică?"
> **Răspunsul tău de inginer:**
>
> "Dacă folosesc any, dezactivez complet verificarea de tipuri din TypeScript. Editorul mă va lăsa să execut o metodă matematică (cum ar fi .toFixed()) pe acel răspuns, chiar și atunci când serverul îmi returnează un String, ceea ce va duce la un crash fatal la runtime.
>
> Dacă folosesc unknown, mențin siguranța arhitecturală. TypeScript mă va obliga (la nivel de compilator) să scriu o condiție de verificare (typeof raspuns === "number") *înainte* de a încerca să accesez datele. unknown mă forțează să scriu teste reziliente, în timp ce any deschide ușa pentru bug-uri ascunse."

2.7 🤖 Mentalitatea de Inginer în Era AI: "Lenea" Asistenților Generativi

> 🌓 **QA vs. AI: Capcana any în Autogenerare**
>
> - **Ce face AI-ul bine:** Modelele de AI înțeleg logica de business și scriu teste E2E foarte rapid.
> - **Capcana (Unde AI-ul eșuează):** Uneori, când îi ceri unui asistent (ex: ChatGPT) să îți construiască o funcție de suport pentru teste, iar structura datelor tale este complexă și nu i-ai descris-o detaliat, AI-ul "devine leneș". Pentru a-și asigura un cod care compilează rapid pe loc, AI-ul va adnota parametrii funcției cu : any.
> - **Cum gândește un inginer QA:** Inginerul respinge codul generat cu any. El știe că AI-ul tocmai a introdus un "Troian" în arhitectura framework-ului de testare. Inginerul va înlocui any cu unknown (obligând AI-ul să scrie verificări defensive) sau va defini el o interfață strictă (vom învăța în Capitolul următor). Tu, ca om, trebuie să fii paznicul strictеții (Strictness Gatekeeper) în fața uneltelor generative!

Capitolul 3: Structuri Avansate - Interfețe (Interfaces) și Tipuri Custom

Obiective de învățare

- 🎯 Înțelegi limitările obiectelor din JavaScript și nevoia de a le adăuga un "contract" strict.
- 🎯 Creezi Alias-uri de tipuri (Type Aliases) și folosești "Union Types" pentru variabile cu mai multe opțiuni permise.
- 🎯 Definești **Interfețe (Interfaces)**, standardul de aur în industrie pentru maparea răspunsurilor API.
- 🎯 Folosești proprietăți opționale (?) și constante stricte (readonly).
- 🎯 Corelezi Interfețele cu scenariile de validare (Assertions) din automatizarea QA.

3.1 Problema: Obiectele haotice din JavaScript

În JavaScript, un Obiect este doar o colecție de proprietăți. Poți să creezi un obiect, să-i adaugi proprietăți noi din mers, să-i ștergi altele sau să te referi la proprietăți care nici măcar nu există (ceea ce îți va returna undefined în tăcere).

```
// Așa arată JS-ul clasic: Fără garanții
let taskNou = {
    titlu: "Scrie testul de Login",
    completat: false
};

// Mai târziu în cod, cineva face o greșeală de tastare:
console.log(taskNou.title); // Printează undefined (trebuia 'titlu')
```

Când testezi un API care îți returnează o listă de 100 de utilizatori, nu poți lăsa codul de

automatizare să ghicească cum arată acel utilizator. Ai nevoie de un contract. Aici intervin Tipurile Custom și Interfețele.

3.2 Tipurile Custom (Type Aliases) și Tipurile Union

Pentru a crea un tip custom, folosim cuvântul cheie **type**. Este ca și cum ai inventa un cuvânt nou în dicționarul TypeScript, pe care îl poți folosi exact la fel cum folosești string sau number.

Unul dintre cele mai puternice instrumente pentru QA este **Union Type (Tipul Uniune)**, marcat prin bara verticală | (care înseamnă SAU).

🤓 ❓ **Testează în TS Playground (www.typescriptlang.org/play):**

```
// Creăm un tip custom care acceptă DOAR 3 stări specifice
type StareTest = "PASSED" | "FAILED" | "SKIPPED";

let rezultatulMeu: StareTest;

rezultatulMeu = "PASSED"; // ☑ Perfect valid!
// rezultatulMeu = "PENDING"; // ❌ EROARE: Type '"PENDING"' is not
assignable to type 'StareTest'.

// Foarte util pentru ID-uri (unele API-uri dau ID-ul ca număr: 12, altele
ca text: "12")
type TestId = string | number;
let idCurent: TestId = 100;
idCurent = "A-100"; // Ambele sunt acceptate fără eroare!
```

3.3 Standardul de Aur pentru Date: Interfețele (Interfaces)

În timp ce type este grozav pentru lucruri simple sau uniuni, când vine vorba de **Obiecte** (cum sunt structurile JSON pe care le primești de la server), industria preferă folosirea cuvântului cheie **interface**.

O Interfață este un șablon, o "matriță" sau un contract. Ea îi spune compilatorului: *"Orice obiect care se pretinde a fi de acest tip TREBUIE să aibă exact aceste chei și aceste tipuri de valori."*

```
// 1. Definim contractul (Numele interfețelor se scriu cu Literă Mare)
```

```
interface Utilizator {
    nume: string;
    email: string;
    varsta: number;
    esteAdmin: boolean;
}

// 2. Creăm un obiect și îl obligăm să respecte contractul
const angajat: Utilizator = {
    nume: "Alex",
    email: "alex@qa.com",
    varsta: 30,
    esteAdmin: true
    // Dacă uiți o proprietate, sau încerci să adaugi una inventată (ex:
adresa), TS îți dă eroare!
};
```

3.4 Magia Interfețelor: ? (Opțional) și readonly

Nu toate datele dintr-un sistem sunt mereu prezente, și nu toate au voie să fie modificate.

- **Proprietățile opționale (?):** Dacă o anumită informație poate să lipsească dintr-un API, îi adăugăm un semn de întrebare înainte de :.
- **Proprietățile Readonly (readonly):** Pentru ID-urile generate de baza de date, care nu au voie să fie suprascrise din greșeală de alt programator.

🤔 ❓ **Testează scenariul QA în TS Playground:**

```
interface ScenariuTest {
    readonly id: number;      // Nu mai poate fi modificat după creare
    titlu: string;
    gradRisc?: "High" | "Low"; // Semnul întrebării înseamnă că poate lipsi
cu totul!
}

const testLogin: ScenariuTest = {
    id: 1,
    titlu: "Validare credențiale",
    // Observă că am omis 'gradRisc' și TypeScript nu mă penalizează!
};

// Încercăm să manipulăm ID-ul (Simulăm o greșeală de cod)
```

```
// testLogin.id = 2; // ❌ EROARE INSTANTĂ: Cannot assign to 'id' because it is a read-only property.
```

3.5 Cum folosim Interfețele în Automatizarea cu Playwright?

Când vei scrie un test E2E care interceptează un răspuns HTTP de la aplicația noastră "Task Tracker" (pe care ai construit-o în Sesiunea 6), codul va arăta astfel:

```
// Definim matrița pentru ce așteptăm de la server
interface TaskTracker {
    id: number;
    title: string;
    completed: boolean;
}

test('Verifică aducerea task-urilor', async ({ request }) => {
    // Cerem datele și le FORȚĂM să fie de tipul Array de TaskTracker
    const response = await request.get('/api/tasks');
    const tasks = await response.json() as TaskTracker[];

    // Acum, când scrii "tasks[0].", VS Code îți va auto-completa "title" și "completed"!
    // Dacă dezvoltatorii schimbă API-ul, testul îți va arăta o eroare roșie imediat.
    expect(tasks[0].title).toBe("Testare validare login");
});
```

3.6 🧠 Challenge de Interviu (Întrebare Tehnică)

> 🧠 **Intervievatorul:** "În TypeScript, poți defini forma unui obiect atât folosind type, cât și folosind interface. Care este diferența dintre ele și pe care o preferi când documentezi răspunsul unui API?"
> **Răspunsul tău de inginer:**
>
> "Diferențele s-au estompat mult în versiunile noi de TS, putând folosi ambele pentru a descrie obiecte. Totuși, **prefer să folosesc interface** pentru răspunsurile API și modelele de date, deoarece interfețele permit 'Declaration Merging' (dacă declari aceeași interfață de două ori, TS le combină inteligent). În schimb, folosesc type strict când am nevoie de un 'Union Type'

(ex: string | number) sau de alias-uri simple pe care o interfață nu le poate exprima direct."

3.7 🤖 Mentalitatea de Inginer în Era AI: "Prompting pe bază de Contract"

🌓 **QA vs. AI: Generarea de Date False (Mock Data)**

- **Problema:** Unul dintre cele mai consumatoare task-uri pentru un QA este crearea de date de test (Mock Data). Dacă îi spui unui AI *"Generează 5 produse de test în JSON"*, va inventa un format din capul lui, pe care aplicația ta probabil îl va respinge.
- **Cum gândește un inginer QA:** Inginerul folosește Interfețele ca **Prompt perfect** pentru AI. El copiază interfața din codul aplicației (interface Produs { id: string; pret: number; stoc?: boolean }) și o introduce în ChatGPT cu mesajul: *"Respectă strict această interfață TypeScript și generează-mi un Array cu 50 de obiecte Mock pentru testele mele E2E"*.
- **Rezultatul:** Asistentul generativ va scuipa un fișier de test impecabil în 3 secunde, cu tipuri de date respectate matematic (fără string-uri puse din greșeală în locul numerelor). Interfețele transformă AI-ul dintr-un generator aleatoriu de text, într-un partener chirurgical.

Capitolul 4: Centrul de Comandă - Configurarea tsconfig.json

Obiective de învățare

- 🎯 Înțelegi de ce a fost inventat fișierul tsconfig.json și cum definește el un "Proiect".
- 🎯 Descoperi setările (flag-urile) fundamentale prin **Misiuni Practice** direct în TS Playground.
- 🎯 Înțelegi de ce strictNullChecks și noImplicitAny sunt salvatoarele vieții unui inginer QA.
- 🎯 Răspunzi la întrebări de interviu legate de configurarea mediului.

4.1 De ce a fost nevoie de tsconfig.json? (Contextul Istoric)

La începuturile TypeScript-ului, dacă voiai să traduci (să compilezi) fișierele tale **.ts** în JavaScript, trebuia să deschizi terminalul și să scrii o comandă manuală, specificând fiecare regulă în parte.

Imaginați-vă un proiect cu zeci de reguli și sute de fișiere. Comanda arăta cam așa:

```
tsc fisier1.ts fisier2.ts fisier3.ts --target ES2015 --module commonjs --strict true --removeComments true --noImplicitAny true...
```

Această abordare era un coșmar imposibil de menținut pentru o echipă. Dacă un programator uita să scrie --strict în terminal, compila un cod plin de bug-uri.

Soluția Microsoft:

> 💡 Au inventat fișierul **tsconfig.json**. Prezența acestui fișier într-un folder îi spune compilatorului: *"Acesta este rădăcina unui Proiect TypeScript. Nu mai aștepta comenzi kilometrice în terminal. Citește pur și simplu regulile din acest fișier de configurare și aplică-le automat pe tot folderul!"*

4.2 Laborator Interactiv: Cum schimbă flag-urile codul tău

*Deschide TS Playground. În meniul de sus, ai un buton numit **"TS Config"**. Acolo vom activa și dezactiva regulile pentru a vedea efectele în timp real.*

Misiunea 1: target (Mașina Timpului)

Această setare dictează "vârsta" JavaScript-ului pe care TS îl va genera la final (în panoul din dreapta).

Execuție:

- Lipește acest cod în panoul din stânga:jav

```
const salut = () => console.log("Hello QA!");
salut(); // Apelăm funcția ca să vedem rezultatul
```

- Deschide meniul **TS Config** și caută **Target**.
- 🟢 **Setează pe ES5 (Cod vechi din 2009):** Uită-te în panoul din dreapta (.JS). Funcția ta săgeată modernă a fost transformată (transpilată) forțat într-o funcție clasică:

```
var salut = function () { return console.log("Hello QA!"); }; salut();
```

Apasă "Run" și vei vedea **"Hello QA!"** în consolă.

- 🟢 **Setează pe ESNext (Cod ultra-modern):** Uită-te în dreapta. Codul a rămas exact cum l-ai scris tu (const și =>), pentru că TS știe că browserele noi suportă această sintaxă.

Misiunea 2: noImplicitAny (Detectorul de lene)

Când e bifat, TS refuză să "ghicească" tipul any și te obligă să scrii cod clar.

Execuție:

- Lipește acest cod în stânga:

```
// Am uitat intenționat să îi dau un tip parametrului 'pret'
function calculeazaPret(pret) {
    return pret * 1.19;
}
console.log(calculeazaPret(100)); // Ar trebui să printeze 119
```

- Deschide **TS Config** și caută **noImplicitAny**.
- 🔘 **Dezactivează-l (debifează):** Codul compilează perfect. Dacă apeși "Run", funcționează. Dar e periculos: mâine un coleg ar putea apela calculeazaPret("O sută") și aplicația va printa NaN (Not a Number).
- 🟢 **Activează-l (bifează):** Instantaneu, cuvântul pret se subliniază cu roșu. Editorul te blochează: *Parameter 'pret' implicitly has an 'any' type.* Nu te lasă să mergi mai departe până nu corectezi codul adăugând (pret: number).

Misiunea 3: strictNullChecks (Salvatorul absolut în QA Automation)

De multe ori căutăm elemente în pagină care s-ar putea să nu existe. Dacă nu verificăm, testul va crăpa la rulare (Runtime Error). Această setare previne asta.

Execuție:

- Lipește acest cod în stânga:

```
// Căutăm un buton FALS care sigur NU există pe pagina TS Playground
const butonSterge = document.querySelector('.buton-fals');

// Încercăm să dăm click direct pe el
butonSterge.click();
```

- Deschide **TS Config** și caută **strictNullChecks**.
- 🔘 **Dezactivează-l:** Editorul nu îți dă nicio eroare. Codul pare "corect". Acum apasă butonul **"Run"**. Consola TS Playground va afișa o eroare fatală: TypeError: Cannot read properties of null (reading 'click'). Testul tău tocmai a picat!
- 🟢 **Activează-l:** Încă din editor, înainte să dai Run, butonSterge este subliniat cu roșu (*Object is possibly 'null'*). TS te forțează să fii un QA bun și să repari logica înainte de rulare:

```
const butonSterge = document.querySelector('.buton-fals');
if (butonSterge) {
    // Doar dacă există pe ecran, dă click. Acum nu mai crapă la Run!
    butonSterge.click();
} else {
    console.log("Butonul nu a fost găsit pe ecran.");
```

```
}
```

Regula Supremă: "strict": true

În fișierele reale tsconfig.json, nu bifezi fiecare opțiune manual. Există o setare supremă numită "strict": true. Dacă o activezi, ea pornește automat (pe fundal) atât noImplicitAny, cât și strictNullChecks, plus alte 5 reguli de siguranță draconice. Este standardul de aur pentru Playwright.

4.3 Exemplu de fișier tsconfig.json minimalist pentru Playwright

Când instalezi framework-ul automat, el va genera acest fișier la rădăcina proiectului tău:

```
{
  "compilerOptions": {
    "target": "ESNext",
    "module": "commonjs",
    "strict": true, // Aici se activează toate regulile învățate mai sus!
    "esModuleInterop": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true
  },
  "include": [
    "tests/**/*", // Spune-i compilatorului să se uite doar în folderul de
teste
    "utils/**/*"
  ]
}
```

4.4 💡 Știați că...?

- **TypeScript te poate ajuta chiar și în JS curat:** Dacă ai un proiect vechi scris în JavaScript clasic și nu vrei să-l transformi în .ts, poți adăuga un fișier tsconfig.json la rădăcină. Pui setările "allowJs": true și "checkJs": true, iar editorul VS Code va începe instantaneu să îți sublinieze greșelile de logică și potențialele bug-uri direct în fișierele tale

> .js! Este cel mai rapid mod de a face audit unui cod vechi.

4.5 🧠 Challenge de Interviu (Întrebare Tehnică)

> **Intervievatorul:** "Am preluat un framework de teste scris de un coleg care a plecat. Mă uit în tsconfig.json și văd că setarea strict este comentată (dezactivată). Ce impact are asta asupra framework-ului și ce se întâmplă cel mai probabil dacă o schimb pe true astăzi, înainte să rulez testele?"
>
> **Răspunsul tău de inginer:**
>
> "Dacă strict este dezactivat, framework-ul permite cod 'leneș' și periculos. Fostul coleg cel mai probabil a declarat funcții fără să specifice tipul parametrilor (folosindu-se de implicit any) și a manipulat elemente din DOM fără să verifice dacă ele chiar există pe ecran (ignorând potențialul de null).
>
> Dacă schimb pe true chiar acum, este foarte probabil ca proiectul să se 'înroșească' brusc în zeci de fișiere direct în editor. Compilatorul mă va bloca și mă va obliga să mă duc la fiecare funcție să adaug tipurile de date lipsă și să verific cu un bloc if fiecare element web înainte să dau click pe el. Deși durează să îl repar, activarea lui strict: true este esențială pentru a scăpa de instabilitate (Flaky Tests)."

4.6 🤖 Mentalitatea de Inginer în Era AI

> 🌓 **QA vs. AI: AI-ul și limitele impuse**
>
> - AI-ul generează cod exact pe măsura mediului în care crede că operează. Dacă îi oferi unui asistent generativ un prompt scurt ("scrie-mi o funcție care dă click pe un meniu"), AI-ul va scrie o versiune scurtă, presupunând că nu folosești reguli stricte (va ignora faptul că meniul ar putea fi null).

- Ca Inginer, trebuie să incluzi mediul arhitectural în Prompt-urile tale: *"Scrie-mi o funcție care accesează meniul. Asumă că proiectul rulează cu **tsconfig strict: true**"*. Această simplă sintagmă îl va forța pe AI să genereze cod defensiv, incluzând verificări de tipuri (typeof) și blocuri de if, scutindu-te pe tine de a repara erorile roșii din VS Code ulterior.

Capitolul 5: Concepte Avansate (Bonus) - Afirmarea Tipurilor (Type Assertions)

Obiective de învățare

- 🎯 Înțelegi limitele inferenței de tip atunci când interacționezi cu Document Object Model (DOM).
- 🎯 Înveți să folosești cuvântul cheie as pentru a forța un tip de date (Type Casting).
- 🎯 Conștientizezi pericolele folosirii abuzive a afirmării tipurilor ("a minți" compilatorul).
- 🎯 Corelezi acest concept cu metodele avansate din Playwright, precum page.evaluate().

5.1 Problema: Când TypeScript știe mai puține decât tine

TypeScript este inteligent, dar nu este clarvăzător. Când scrii cod care interacționează cu pagina web, TS se uită la funcția document.querySelector('.clasa-mea') și știe că aceasta va returna un **Element** generic (sau null).

Problema? Un Element generic nu are o proprietate numită .value (aceea aparține doar câmpurilor de text) și nu are o proprietate .disabled (aceea aparține butoanelor).

🤔 ❓ **Testează EROAREA în TS Playground:**

Lipește codul de mai jos în panoul din stânga. **Nu apăsa Run!** Uită-te doar la avertismentul vizual.

```
// Găsim un element pe pagină. TS crede că este un "Element" generic.
const inputEmail = document.querySelector('.email-field');

// Încercăm să scriem un text în el
if (inputEmail) {
    inputEmail.value = "test@qa.com";
}
```

Dacă ai setarea strict: true activată, cuvântul value se va înroși!

Eroarea TS: *Property 'value' does not exist on type 'Element'.* TypeScript refuză să te lase să accesezi .value pentru că, din punctul lui de vedere, acel element ar putea fi la fel de bine un sau un <h1> care nu au valoare editabilă!

5.2 Salvarea: Cuvântul cheie as (Type Casting)

Aici intervine **Afirmarea Tipurilor** (Type Assertion). Prin folosirea cuvântului cheie as, tu îi transmiți compilatorului un mesaj clar: *"Știu că tu crezi că acesta este doar un Element oarecare, dar eu, ca inginer, am inspectat pagina și îți garantez că este un element de tip Input. Tratează-l ca atare!"*

🕵️ ❓ Testează EXECUȚIA în TS Playground:

Șterge codul vechi și lipește-l pe acesta. Pentru ca testul să meargă în Playground, vom crea noi input-ul mai întâi! Apasă **Run** și privește Consola!

```
// 1. (Setup) Injectăm temporar un input în pagina Playground-ului pentru
a-l putea găsi
const inputFals = document.createElement('input');
inputFals.className = 'email-field';
document.body.appendChild(inputFals);

// 2. (Testul) Căutăm input-ul și forțăm tipul folosind "as
HTMLInputElement"
const inputGasit = document.querySelector('.email-field') as
HTMLInputElement;

if (inputGasit) {
    // Acum TS îți oferă auto-completare, acceptă comanda și putem vedea
rezultatul!
    inputGasit.value = "test@qa.com";
    console.log("Succes! Am reușit să setăm valoarea:", inputGasit.value);
}
```

Cele mai folosite tipuri de DOM în QA:

- as HTMLInputElement (pentru <input>, dă acces la .value, .checked)
- as HTMLButtonElement (pentru <button>, dă acces la .disabled)
- as HTMLSelectElement (pentru meniul de tip Dropdown <select>)

5.3 Pericolul ascuns: Mințirea Compilatorului

Afirmarea tipurilor este o armă cu două tăișuri. Când folosești as, **tu oprești protecția TypeScript pentru acea linie de cod**. Compilatorul te va crede pe cuvânt.

Dacă te înșeli, aplicația va da crash la rulare, exact ca în vremurile JavaScript-ului pur.

```
// Aici mințim intenționat compilatorul. Selectăm un buton, dar spunem că e
Input.
const butonTrimite = document.querySelector('.btn-submit') as
HTMLInputElement;

// TypeScript nu dă nicio eroare aici, pentru că l-am forțat!
// Dar la rularea în browser, acest cod va eșua tăcut sau va avea
comportament haotic.
butonTrimite.value = "Hacked";
```

Regula de Aur a Seniorilor: Folosește as doar atunci când ai un motiv tehnic absolut valid (cum ar fi selecția din DOM) și ești 100% sigur de tipul de date. Nu folosi as any doar pentru a scăpa rapid de o eroare roșie din editor!

5.4 Legătura cu Playwright: Executarea codului în Browser

Când folosești Playwright în mod normal (await page.click()), nu ai nevoie de as HTMLButtonElement, pentru că framework-ul abstractizează interacțiunea pentru tine.

Totuși, în scenarii complexe de testare (cum ar fi extragerea tuturor link-urilor dintr-un tabel invizibil sau preluarea unor variabile ascunse din consolă), vei folosi comanda **page.evaluate()**. Această comandă ia o bucată din codul tău TypeScript și o injectează direct în browserul Chrome, rulând-o acolo.

```
// Exemplu real dintr-un test avansat Playwright
const statutButon = await page.evaluate(() => {
    // Aici suntem în interiorul browserului! Trebuie să folosim TypeScript
pur.
    const buton = document.querySelector('#btn-finalizare') as
HTMLButtonElement;
    return buton.disabled;
});

expect(statutButon).toBe(true);
```

Fără cunoașterea cuvântului cheie as, nu vei putea scrie scripturi evaluate complexe, deoarece editorul VS Code va refuza să îți compileze testul.

5.5 🤖 Mentalitatea de Inginer în Era AI: Asertările

Oarbe

> 🌓 **QA vs. AI: Abuzul de Type Casting**
>
> - **Capcana (Unde AI-ul eșuează):** Uneori, un asistent AI va genera un cod unde face conversii de date forțate pentru a scăpa de erorile stricte, de exemplu: const date = raspunsAPI as any as Utilizator;. Aceasta este o practică extrem de toxică, denumită "Double Casting" (Asertare dublă), care ascunde complet greșelile de structură de la server.
> - **Cum gândește un inginer QA:** Inginerul respinge din start asertările duble sau folosirea lui as peste răspunsurile API nescuzabile. El preferă să folosească instrumente de validare a schemelor de date (ex: librăria Zod) sau să scrie *Type Guards* (funcții de verificare manuală), rezervând cuvântul as exclusiv pentru selecțiile sigure din DOM (HTMLElement).

Capitolul 6: Tema Sesiunii 7 & Extra Practice (Spre nivelul de Senior)

6.1 Tema pentru Acasă (Proiect Sesiunea 7) & Platforma QualiAdept Cloud Evaluation

> 🎯 **Obiectivul Temei:** Refactorizarea aplicației "Task Tracker" prin adăugarea primului strat de tipizare statică TypeScript. Vei învăța să securizezi variabilele, modelele de date și funcțiile, eliminând riscurile erorilor de tip undefined și interzicând utilizarea nesigură a tipului any.

🏢 Scenariul de Business (Task-ul de business):

> 📝 Echipa ta a livrat cu succes versiunea 1.0 a aplicației "Task Tracker" în Sesiunea 6. Înaintea începerii scrierii testelor automate de Playwright din Sesiunea 9, Lead Architect-ul și echipa de QA au decis ca **întregul codebase să fie migrat către TypeScript** pentru a garanta stabilitatea la nivel Enterprise.
>
> Sarcina ta este să transformi scriptul de logică într-un modul TypeScript securizat (app.ts). Vei introduce tipuri explicite pentru toate datele manipulate (primitive, array-uri, tipuri de uniune pentru statusuri) și vei elimina orice utilizare a periculosului any.
>
> ***Notă pentru execuție**: Vei scrie codul în fișierul **app.ts** (care preia și extinde logica din fostul app.js). În index.html, acesta va fi conectat prin <script src="app.ts"></script> (presupunând că platforma/bundler-ul face compilarea în spate, sau folosești TS Playground pentru a obține JS-ul).*

📋 Cerințe Tehnice (Acceptance Criteria pentru platforma de validare):

- **Tipizarea Primitivelor & Variabilelor de Configurare:**
  - Declară explicit tipurile primitive pentru variabilele de configurare ale aplicației:
    - apiUrl: string (URL-ul API-ului extern)
    - maxTasksLimit: number (limita de 5 task-uri)
    - isAppReady: boolean (indicator de stare a aplicației)
- **Tipul de Uniune (Union Type) pentru Statusul Task-ului:**
  - Pentru a evita greșelile de tipar la setarea stărilor, definește un Type Alias numit TaskStatus.
  - Acesta trebuie să accepte strict una dintre cele trei valori literale: 'todo' | 'in-progress' | 'completed'.
- **Modelul de Date Tipizat (Type Alias TaskItem):**
  - Definește un Type Alias numit TaskItem care să descrie cu strictețe structura unui task în memorie:
    - id: number;

    - title: string;
    - status: TaskStatus;
    - completed: boolean;
    - priority?: 'low' | 'medium' | 'high'; *(proprietate opțională)*
  - Declară array-ul principal listaTaskuri tipizat explicit ca o colecție de astfel de obiecte: TaskItem[] (sau Array<TaskItem>).
- **Semnăturile Funcțiilor & Eliminarea lui any (Zero Any Policy):**
  - **Funcția de adăugare task (proceseazaTaskNou):**
    - Parametrul de intrare taskObj trebuie să fie tipizat ca TaskItem.
    - Valoarea returnată de funcție trebuie să fie explicit de tipul: boolean.
  - **Funcția de afișare/raportare (afiseazaTask):**
    - Primește parametrul id: number și returnează explicit void (deoarece doar afișează în consolă fără să returneze o valoare).
  - **Regulă strictă QA:** Este strict interzisă utilizarea tipului : any. Orice apariție a lui any va duce la eșuarea validării automate!
- **Funcție Asincronă Tipizată (Fetch API & Promise):**
  - Funcția incarcaTaskuriInitiale trebuie să specifice explicit valoarea de retur asincronă: Promise<void>.
  - Datele convertite cu await response.json() trebuie asertate la tipul corect folosind type assertion (as TaskItem[]).

💡 Analogie QA Automation (De ce este vital pentru Playwright):

În Playwright vei lucra 100% cu TypeScript:

- Când cauți un element pe pagină, metoda returnează un Locator strict tipizat.
- Când apelezi acțiuni (click, fill, goto), toate returnează un Promise<void>.
- Când creezi scenarii de test bazate pe date (Data-Driven Testing), definirea unor tipuri precum TaskItem te va împiedica să testezi câmpuri inexistente, eliminând erorile umane înainte de rularea primului test!

6.2 Provocarea Bonus (Opțional): Siguranță absolută cu unknown

Scenariul: Ai asertat răspunsul de la server cu as TaskItem[]. Deși e o practică comună, în

echipele super-stricte, mințim compilatorul dacă serverul schimbă brusc formatul datelor.

Misiunea ta:

Schimbă asertarea. Salvează rezultatul de la await response.json() într-o variabilă de tip unknown. Apoi, scrie o condiție (Type Guard) care verifică dacă datele primite reprezintă un Array. Doar dacă este adevărat, transferă datele în listaTaskuri.

6.3 🔑 Cheile de Rezolvare (Soluții & Explicații)

Nu trișa! Încearcă să scrii codul singur înainte de a verifica soluția. Această rezolvare va trece 100% de robotul de validare QualiAdept.

Soluție Tema pentru Acasă (app.ts):

```
// 1. Tipizarea Primitivelor
const apiUrl: string = 'https://jsonplaceholder.typicode.com/todos';
const maxTasksLimit: number = 5;
let isAppReady: boolean = false;

// 2. Tipul de Uniune (Union Type)
type TaskStatus = 'todo' | 'in-progress' | 'completed';

// 3. Modelul de Date Tipizat
type TaskItem = {
    id: number;
    title: string;
    status: TaskStatus;
    completed: boolean;
    priority?: 'low' | 'medium' | 'high'; // Proprietate opțională
};

// Declararea colecției tipizate
let listaTaskuri: TaskItem[] = [];

// 4. Semnăturile Funcțiilor (Zero Any Policy)
const proceseazaTaskNou = (taskObj: TaskItem): boolean => {
    if (!taskObj.title) {
        console.error("Eroare: Task-ul trebuie să aibă un titlu!");
        return false;
    }
```

```
    listaTaskuri.push(taskObj);
    console.log(`Task-ul "${taskObj.title}" a fost adăugat.`);
    return true;
};

const afiseazaTask = (id: number): void => {
    const taskGasit = listaTaskuri.find(task => task.id === id);
    if (taskGasit) {
        console.log(`[${taskGasit.status.toUpperCase()}]
${taskGasit.title}`);
    } else {
        console.log(`Task-ul cu ID ${id} nu a fost găsit.`);
    }
    // Returnează implicit void
};

// 5. Funcție Asincronă Tipizată
const incarcaTaskuriInitiale = async (): Promise<void> => {
    try {
        const response = await fetch(`${apiUrl}?_limit=${maxTasksLimit}`);

        if (!response.ok) {
            throw new Error("Eroare la conectarea cu API-ul");
        }

        // Asertăm tipul datelor primite
        const dateServer = await response.json() as TaskItem[];

        // Populăm lista noastră
        listaTaskuri = dateServer;
        isAppReady = true;

        console.log(`Au fost încărcate ${listaTaskuri.length} task-uri.`);

    } catch (error) {
        console.error("Sistemul a întâmpinat o problemă:", error);
    }
};

// Simulăm o execuție
incarcaTaskuriInitiale().then(() => {
    // După ce s-au încărcat datele, adăugăm un task nou strict tipizat
```

```
    const testNou: TaskItem = {
        id: 999,
        title: "Rulează suita Playwright",
        status: "todo",
        completed: false,
        priority: "high"
    };

    proceseazaTaskNou(testNou);
    afiseazaTask(999);
});
```

🎉 **Felicitări!** Acesta este primul tău cod valid de TypeScript. Ești pregătit arhitectural să abordezi cele mai avansate concepte în Playwright!

Workbook Sesiunea 7 - TypeScript Core

> 📈 **Scopul:** Construirea "memoriei musculare" pentru Playwright. Toate exercițiile de mai jos sunt **scripts complete**. Ele conțin declararea tipurilor, instanțierea lor, apelarea metodelor și printarea rezultatelor (pentru a putea fi verificate).
>
> 🛠️ **Setup:** Deschide **TypeScript Playground** (cu strict: true activat). Copiază fiecare bloc de cod, lipește-l în stânga și apasă **Run**. Verifică panoul "Logs" (Consola) din dreapta!

📍 Exercițiul 1: Funcții, Primitive și Type Inference

Scenariu: O funcție de validare care calculează dacă un test a durat prea mult. Funcția este tipizată strict și apoi **apelată**.

```
// 1. Definim funcția cu parametri strict tipizați și retur explicit
const valideazaTimpExecutie = (numeTest: string, timpMilisecunde: number,
limitaMax: number): boolean => {
    console.log(`[RULARE] Testul "${numeTest}" a durat
${timpMilisecunde}ms.`);
    return timpMilisecunde <= limitaMax;
};

// 2. APELĂM METODELE și testăm scenariile (Pass și Fail)
const testLogin = valideazaTimpExecutie("Login cu succes", 1200, 2000);
const testCheckout = valideazaTimpExecutie("Checkout card", 3500, 2000);

// 3. Verificăm rezultatele (Assertions)
console.log(`Rezultat Login: ${testLogin ? "✅ PASSED" : "❌ FAILED"}`);
console.log(`Rezultat Checkout: ${testCheckout ? "✅ PASSED" : "❌
FAILED"}`);
```

📍 Exercițiul 2: Union Types & Literal Types (Opțiuni Restricționate)

Scenariu: Simulează o funcție care dă click pe un buton, dar acel buton poate avea doar 3 stări specifice. Nu acceptăm orice string.

```
// 1. Definim tipurile permise (Doar aceste 3 cuvinte exacte!)
type StareButon = "activ" | "dezactivat" | "ascuns";

// 2. Funcție utilitară pentru testare
const daClickPeButon = (selector: string, stare: StareButon): void => {
    if (stare === "dezactivat") {
        console.log(`❌ EROARE: Nu pot da click pe ${selector} pentru că
este dezactivat!`);
        return;
    }
    if (stare === "ascuns") {
        console.log(`❌ EROARE: Selectorul ${selector} nu este vizibil pe
ecran!`);
        return;
```

```
    }
    console.log(`☑ Click efectuat cu succes pe ${selector}.`);
};

// 3. APELĂM METODA în diferite contexte
daClickPeButon("#btn-submit", "activ");
daClickPeButon("#btn-cancel", "dezactivat");

// ✖ Daca decomentezi linia de mai jos, TS dă eroare înainte de rulare!
// daClickPeButon("#btn-fake", "inexistent");
```

📍 Exercițiul 3: Siguranța cu unknown (Type Guards)

Scenariu: Simulăm citirea unui răspuns de la un API terț unde nu avem garanția structurii, așa că verificăm tipul *înainte* de a apela metode.

```
// 1. O funcție care primește date nesigure (ex: din LocalStorage)
const proceseazaDateMisterioase = (date: unknown): void => {
    console.log("[PROCESARE] Analizăm datele primite...");

    // TYPE GUARD: Verificăm sigur dacă e text
    if (typeof date === "string") {
        console.log(`Este Text! Versiune formatată:
${date.toUpperCase()}`);
    }
    // TYPE GUARD: Verificăm sigur dacă e număr
    else if (typeof date === "number") {
        console.log(`Este Număr! Prețul cu TVA: ${date * 1.19}`);
    }
    else {
        console.log("⚠ Format necunoscut. Se ignoră.");
    }
};

// 2. APELĂM FUNCȚIA testând siguranța compilatorului
proceseazaDateMisterioase("token-secret-de-la-server");
proceseazaDateMisterioase(250);
proceseazaDateMisterioase(true); // Va intra pe ramura de format necunoscut
```

📍 Exercițiul 4: Interfețe și Array-uri (Manipularea datelor)

Scenariu: Avem un contract (interface) pentru utilizatori. Vom crea o listă și o metodă care filtrează utilizatorii admini.

```
// 1. Definim contractul (Interface)
interface UtilizatorTest {
    readonly id: number;
    username: string;
    esteAdmin: boolean;
}

// 2. Creăm o bază de date (Array tipizat)
const bazaDeDateQA: UtilizatorTest[] = [
    { id: 101, username: "john_doe", esteAdmin: false },
    { id: 102, username: "super_qa", esteAdmin: true },
    { id: 103, username: "dev_lead", esteAdmin: true }
];

// 3. Metodă pentru a extrage doar adminii
const gasesteAdmini = (lista: UtilizatorTest[]): UtilizatorTest[] => {
    return lista.filter(user => user.esteAdmin === true);
};

// 4. APELĂM ȘI VERIFICĂM
const adminiGasiti = gasesteAdmini(bazaDeDateQA);

console.log(`S-au găsit ${adminiGasiti.length} utilizatori Admin.`);
adminiGasiti.forEach(admin => {
    console.log(`- Cont de acces privilegat: ${admin.username} (ID:
${admin.id})`);
});
```

📍 Exercițiul 5: Clase și Încapsulare (Page Object

Model)

Scenariu: Acesta este nucleul pentru Playwright! Creăm o clasă care reprezintă o pagină web. Încapsulăm selectorii și expunem doar metodele publice de test.

```
// 1. Definim Clasa
class PaginaLogin {
    // Selectoare private (Protejate de restul framework-ului)
    private selectorUser = "#input-email";
    private selectorParola = "#input-password";
    private butonLogin = "#btn-submit";

    // Variabilă care păstrează URL-ul
    public url: string;

    constructor(urlStart: string) {
        this.url = urlStart;
        console.log(`[POM] S-a inițializat pagina de login pentru:
${this.url}`);
    }

    // 2. METODE PUBLICE (Acțiuni care pot fi apelate din test)
    public scrieCredențiale(user: string, pass: string): void {
        console.log(`Se tastează '${user}' în
[this.selectorUser]...`);console.log(`SetasteazăparolaîncastratăîN[{this.se
lectorParola}]...`);
    }

    public daClickPeLogin(): void {
        console.log(`Click pe selectorul: [${this.butonLogin}]`);
    }
}

// 3. EXECUTĂM TESTUL (Instanțiem și apelăm metodele secvențial)
const loginPage = new PaginaLogin("https://app.ro/login");

console.log("--- Începe Scenariul E2E ---");
loginPage.scrieCredențiale("admin@qa.com", "Parola123!");
loginPage.daClickPeLogin();
console.log("--- Scenariu Finalizat ---");
```

```
// ❌ Eroare: loginPage.selectorUser = ".alt-clasa"; (Proprietate privată!)
```

📍 Exercițiul 6: Moștenirea (Extends & Super)

Scenariu: Orice pagină din aplicația ta are un Navbar. În loc să scriem metodele de meniu în fiecare clasă, creăm o BasePage din care moștenim funcționalitățile.

```
// 1. Clasa Părinte (Generală)
class BasePage {
    constructor(protected pageName: string) {}

    public verificaTitlu(): void {
        console.log(`[BasePage] Verificăm dacă titlul paginii conține
'${this.pageName}'`);
    }

    public logout(): void {
        console.log(`[BasePage] S-a dat click pe 'Delogare' din Navbar-ul
principal.`);
    }
}

// 2. Clasa Copil (Specifică, extinde Părintele)
class DashboardPage extends BasePage {
    constructor() {
        super("Dashboard Principal"); // Apelează constructorul din
BasePage
    }

    public genereazaRaport(): void {
        console.log("[DashboardPage] Se descarcă raportul de testare...");
    }
}

// 3. APELĂM METODELE MOȘTENITE
const myDashboard = new DashboardPage();

// Metodă venită gratuit de la părinte!
```

```
myDashboard.verificaTitlu();

// Metoda specifică copilului!
myDashboard.genereazaRaport();

// Metodă venită gratuit de la părinte!
myDashboard.logout();
```

🎯 Exercițiul 7: Manipularea DOM-ului și Type Casting (as)

Scenariu: Simulează un test intern în browser (page.evaluate). Trebuie să forțăm compilatorul să știe că un element găsit este de fapt un HTMLInputElement.

```
// 1. (SETUP): Creăm input-ul direct în memoria Playground-ului ca să poată
fi testat live
const DOM = document.body;
DOM.innerHTML = `<input type="text" id="email-test" value="vechi@qa.com">
                 <button id="save-btn" disabled>Salvează</button>`;

// 2. (TESTUL REAL): Type Casting
// Folosim "as HTMLInputElement" ca să avem acces la proprietatea .value
const inputField = document.querySelector('#email-test') as
HTMLInputElement;

// Folosim "as HTMLButtonElement" ca să avem acces la proprietatea
.disabled
const submitBtn = document.querySelector('#save-btn') as HTMLButtonElement;

// 3. APELĂM METODE PE DOM ȘI VERIFICĂM
if (inputField && submitBtn) {
    console.log(`Valoarea INIȚIALĂ a input-ului este:
${inputField.value}`);
    console.log(`Butonul de salvare este DEZACTIVAT? :
${submitBtn.disabled}`);

    // Acțiune: Scriem text nou și activăm butonul
```

```
    inputField.value = "nou@qa.com";
    submitBtn.disabled = false;

    console.log(`Valoarea NOUĂ a input-ului este: ${inputField.value}`);
    console.log(`Butonul de salvare este DEZACTIVAT? :
${submitBtn.disabled}`);
}
```

🎉 **Acesta este adevăratul cod pe care îl vei scrie zi de zi!** Cu aceste exemple nu doar că am învățat teoria, dar am *instanțiat* clase, am *apelat* funcții polimorfice, am *transmis* argumente respectând contractele stricte și am *validat* vizual rezultatele manipulării DOM-ului!